Université Grenoble Alpes

Licence Informatique – Systèmes et Réseaux

Introduction aux réseaux

Cours de synthèse à partir du TD

Couches, encapsulation, adressage, routage, performances et transfert de fichiers

Support pédagogique structuré pour comprendre le TD de bout en bout

Ce document transforme le TD d'introduction aux réseaux en un **véritable support de cours**. L'objectif n'est pas seulement de donner les réponses, mais de construire les idées qui permettent de **comprendre**, **raisonner** et **résoudre** les exercices de manière autonome.

Rédigé à partir du sujet de TD, du corrigé local et des supports de cours présents dans le répertoire.

Mars 2026

Table des matières

1 Pourquoi les réseaux existent-ils ? 2

2 L'architecture en couches 2

2.1 Pourquoi découper le réseau en couches ? . . . 2
2.2 Modèle OSI et pile Internet . . . 2
2.3 Encapsulation : comment un message descend dans la pile . . . 3

3 De la machine au bon programme : IP, ports et routage 4

3.1 Adresses locales, adresses globales et ports . . . 4
3.2 Comment le système distingue TCP, UDP, ICMP... . . . 4
3.3 Routage : faire passer un paquet d'un réseau à l'autre . . . 4
3.4 Que fait un routeur s'il ne connaît pas la destination ? . . . 5
3.5 Peut-on tout faire avec n'importe quels ports ? . . . 5

4 Ce que l'on observe vraiment sur le lien 5

5 Performances du réseau : débit, latence et coût des sauts 6

5.1 Deux grandeurs à connaître absolument . . . 6
5.2 Petit message ou gros fichier : le facteur limitant n'est pas le même . . . 6
5.3 Débit critique : quand propagation et transmission sont égales . . . 7
5.4 L'impact des routeurs intermédiaires . . . 7
5.5 Store-and-forward contre cut-through . . . 7
5.6 Segmentation et pipeline . . . 8

6 Focus application : transfert de fichiers et architecture client-serveur 9

6.1 Ce que fait réellement un protocole de transfert . . . 9
6.2 Serveur itératif ou serveur concurrent ? . . . 9

7 Méthode générale pour résoudre les exercices du TD 10

8 Fiche de synthèse 10

1 Pourquoi les réseaux existent-ils ?

Un réseau informatique est un ensemble de **machines**, de **liens de communication** et de **protocoles** qui permet d'échanger des informations. Mais un réseau n'est pas une fin en soi : il sert avant tout à rendre des **services** accessibles à distance.

Quelques exemples classiques :

— consulter une page Web ;
— transférer un fichier ;
— envoyer un courrier électronique ;
— accéder à un serveur ou à une base de données ;
— faire dialoguer deux programmes client et serveur.

Dans toute la suite, il faut garder en tête la distinction suivante :

— le **réseau** transporte l'information ;
— les **applications** exploitent ce transport pour fournir un service.

Objectifs du chapitre

— comprendre pourquoi on organise les réseaux en couches ;
— savoir distinguer **machine**, **interface**, **adresse IP**, **port** et **socket** ;
— expliquer comment un message est acheminé d'une application vers une autre ;
— maîtriser les formules de base sur le débit, la latence et le transfert en plusieurs sauts.

2 L'architecture en couches

2.1 Pourquoi découper le réseau en couches ?

Un réseau est trop complexe pour être pensé comme un bloc unique. On le découpe donc en **couches**, chacune ayant un rôle précis. Cette idée apporte trois bénéfices majeurs :

— **maîtrise de la complexité** : chaque couche résout un problème bien identifié ;
— **modularité** : on peut faire évoluer une couche sans tout redéfinir ;
— **clarté des interfaces** : chaque couche fournit un service à la couche au-dessus.

Il faut distinguer deux notions :

— une **interface** décrit **ce qu'un service offre** ;
— un **protocole** décrit **comment deux entités d'un même niveau coopèrent** pour rendre ce service.

2.2 Modèle OSI et pile Internet

Le modèle OSI comporte sept couches. En pratique, l'Internet fonctionne surtout avec une pile à cinq couches :

Couche Internet	Unité	Rôle principal	Exemples
Application	message	fournit un service directement utile au programme	HTTP, DNS, FTP, SSH
Transport	segment / datagramme	communication de bout en bout entre processus, ports, fiabilité éventuelle	TCP, UDP
Réseau	paquet	adressage logique et routage entre réseaux distincts	IP
Liaison	trame	communication sur un lien local, adressage MAC, détection d'erreurs	Ethernet, Wi-Fi
Physique	bit	transmission du signal sur le support matériel	cuivre, fibre, radio

Dans la pile Internet, les fonctions *session* et *présentation* du modèle OSI sont en général absorbées par la couche application.

À retenir

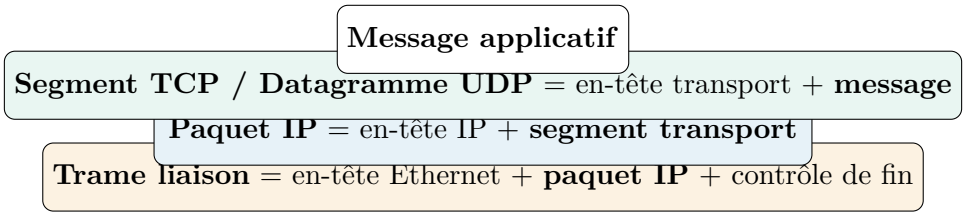
— La couche **liaison** travaille **lien par lien**.

— La couche **réseau** permet de traverser **plusieurs réseaux**.

— La couche **transport** permet de livrer les données au **bon processus** grâce aux ports.

2.3 Encapsulation : comment un message descend dans la pile

Quand une application envoie des données, chaque couche ajoute ses propres informations de contrôle. C'est le principe d'**encapsulation**.



À la réception, le mécanisme inverse est appliqué : on parle de **décapsulation**. Chaque couche lit l'en-tête qui la concerne, retire les informations qui lui sont propres puis transmet le contenu à la couche supérieure.

Exemple : transfert de fichier

Si un client récupère un fichier via FTP :

— l'application construit une requête ou des données de fichier ;

— TCP découpe le flot en segments et numérote les échanges ;

— IP choisit l'adressage logique pour joindre la machine distante ;

— Ethernet ou le Wi-Fi transporte les trames sur le lien local du moment.

3 De la machine au bon programme : IP, ports et routage

3.1 Adresses locales, adresses globales et ports

Pour qu'une communication arrive au bon endroit, il faut distinguer trois niveaux d'identification :

— l'**adresse MAC** identifie une interface sur un **réseau local** ;
— l'**adresse IP** identifie une machine sur un ensemble de **réseaux interconnectés** ;
— le **port** identifie un **processus** ou un service sur cette machine.

Une machine peut offrir plusieurs services en même temps. L'adresse IP seule ne suffit donc pas. Il faut y ajouter un numéro de port.

Exemple : socket

Une **socket** désigne une extrémité de communication. En pratique, on la repère par un triplet du type :

(protocole de transport, adresse IP, port)

Par exemple : `(TCP, 216.239.39.99, 80)` pour un serveur Web.

3.2 Comment le système distingue TCP, UDP, ICMP...

Lorsqu'une machine reçoit un paquet IP qui lui est destiné, le système d'exploitation lit un champ de l'en-tête IP :

— en IPv4 : le champ **Protocol** ;
— en IPv6 : le champ **Next Header**.

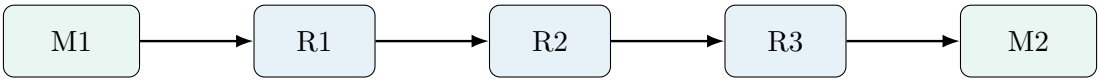
Ce champ indique le protocole encapsulé dans la charge utile :

— `6` pour TCP ;
— `17` pour UDP ;
— `1` pour ICMP en IPv4.

Une fois le protocole identifié, le noyau regarde ensuite le **numéro de port** pour livrer les données à la bonne application.

3.3 Routage : faire passer un paquet d'un réseau à l'autre

Un **routeur** ne s'intéresse pas au contenu applicatif. Son rôle est de choisir le **prochain saut** pour faire progresser le paquet vers sa destination.



Sur le chemin $M1 \to R1 \to R2 \to R3 \to M2$:

— il y a **4 liens** ;
— le paquet traverse donc **8 interfaces réseau** ;
— si l'on compte la décision initiale prise par $M1$, il y a **4 décisions de prochain saut**.

Point d'attention

Certaines corrections ne comptent que les **routeurs intermédiaires** et annoncent 3 décisions de routage. C'est une convention plus restrictive. Pour raisonner proprement, il est préférable de compter aussi le choix initial du prochain saut par la machine source.

3.4 Que fait un routeur s'il ne connaît pas la destination ?

Deux cas sont possibles :

1. s'il existe une **route par défaut**, il envoie le paquet vers ce prochain saut ;
2. sinon il **supprime le paquet**, souvent en envoyant en retour un message ICMP d'erreur.

Le routeur ne garde pas le paquet "en attente" en espérant qu'une meilleure route apparaisse plus tard : ce n'est pas le comportement normal d'IP.

3.5 Peut-on tout faire avec n'importe quels ports ?

Non. Il faut raisonner avec précision sur les **protocole de transport** et **identifiants de connexion**.

Cas	Situation	Possible ?	Raisonnement
1	Un côté parle TCP et l'autre UDP	Non	TCP et UDP sont deux protocoles distincts ; les deux extrémités doivent parler le même protocole.
2	Deux connexions TCP simultanées avec exactement le même quadruplet $(IP_{src}, port_{src}, IP_{dst}, port_{dst})$	Non	Elles seraient indiscernables.
3	Un même client ouvre deux connexions TCP vers le même serveur avec deux ports locaux différents	Oui	Les quadruplets sont différents.
4	Utiliser le même numéro numérique en TCP et en UDP pour deux applications différentes	Oui	Les espaces de ports TCP et UDP sont séparés.

À retenir

Le port ne suffit jamais seul à décrire une communication. Pour TCP, on identifie une connexion par le **quadruplet** :

(IP source, port source, IP destination, port destination)

4 Ce que l'on observe vraiment sur le lien

Question classique : si l'on observe les **premiers octets** transmis lors d'un transfert de fichier, voit-on directement les octets du fichier ? La réponse est **non**.

Avant d'apercevoir la charge utile du fichier, on observe généralement :

1. éventuellement des trames de résolution d'adresse locale, par exemple ARP ;
2. des en-têtes de liaison ;
3. des en-têtes IP ;
4. des en-têtes TCP ou UDP ;
5. parfois des messages de contrôle applicatifs ;
6. enfin seulement, les données du fichier.

Autrement dit, **sur le fil, on voit d'abord l'encapsulation**. Les octets du fichier n'arrivent qu'après.

Exemple : transfert TCP

Pour un transfert TCP de fichier, il est fréquent d'observer successivement :

— une résolution d'adresse locale ;
— l'établissement de connexion TCP (`SYN`, `SYN-ACK`, `ACK`) ;
— des segments transportant des données applicatives ;
— des accusés de réception TCP ;
— puis la fermeture de la connexion.

5 Performances du réseau : débit, latence et coût des sauts

5.1 Deux grandeurs à connaître absolument

Les performances d'un réseau se lisent d'abord avec deux notions :

— le **débit** : quantité d'information transmise par unité de temps, en bit/s ;
— la **latence** : temps écoulé entre l'émission et la réception.

La latence totale se décompose en :

$$\text{latence} = \text{temps d'émission} + \text{temps de propagation} + \text{temps d'attente}$$

avec :

$$t_{\text{émission}} = \frac{\text{taille}}{\text{débit}} \qquad \text{et} \qquad t_{\text{propagation}} = \frac{\text{distance}}{\text{vitesse de propagation}}$$

À retenir

— Le **temps d'émission** dépend de la **taille** du message et du **débit**.
— Le **temps de propagation** dépend de la **distance** et du **support physique**.
— La **latence de base** correspond à ce qui est incompressible avant la réception du premier bit.

5.2 Petit message ou gros fichier : le facteur limitant n'est pas le même

Considérons un réseau de longueur $L = 10^7$ m, de débit $D = 10^6$ bit/s et de vitesse de propagation $V = 2 \times 10^8$ m/s.

$$t_{\text{propagation}} = \frac{10^7}{2 \times 10^8} = 0{,}05 \text{ s} = 50 \text{ ms}$$

Pour un message de 1000 bits :

$$t_{\text{émission}} = \frac{1000}{10^6} = 1 \text{ ms}$$

Pour un fichier de 10^6 bits :

$$t_{\text{émission}} = \frac{10^6}{10^6} = 1 \text{ s}$$

Conclusion :

— pour de **petits messages interactifs**, la propagation domine : 50 ms contre 1 ms ;
— pour un **gros transfert**, le débit domine : 1 s contre 50 ms.

Point d'attention

Le mot "latence" est souvent utilisé de manière vague. Dans les exercices, il faut toujours se demander : **parle-t-on du temps avant le premier bit ou du temps avant le dernier bit ?** Les réponses ne sont pas les mêmes.

5.3 Débit critique : quand propagation et transmission sont égales

Pour un paquet de P octets sur une liaison de longueur L et de vitesse V, on cherche le débit D tel que :

$$\frac{L}{V} = \frac{8P}{D}$$

On obtient :

$$D = \frac{8PV}{L}$$

Exemple : paquet de 100 octets sur 40 km

Avec $L = 40$ km, $P = 100$ octets et $V = 2 \times 10^5$ km/s :

$$D = \frac{8 \times 100 \times 2 \times 10^5}{40} = 4 \times 10^6 \text{ bit/s}$$

Donc le débit recherché est **4 Mbit/s**.
Le temps de propagation vaut alors :

$$\frac{40}{2 \times 10^5} = 2 \times 10^{-4} \text{ s} = 0{,}2 \text{ ms}$$

5.4 L'impact des routeurs intermédiaires

Un paquet qui traverse plusieurs nœuds n'est pas seulement ralenti par la propagation. Il peut aussi subir :

— des **délais de commutation** ;
— des **temps d'attente** dans les files ;
— des **temps de retransmission** sur chaque lien.

Prenons $N = 50$ routeurs, une distance totale $d = 5000$ km, une vitesse $V = 2 \times 10^8$ m/s et un délai de commutation $t = 10$ μs.

$$t_{\text{propagation}} = \frac{5000 \times 10^3}{2 \times 10^8} = 25 \text{ ms}$$

$$t_{\text{commutation}} = Nt = 50 \times 10 \ \mu s = 0{,}5 \text{ ms}$$

Ici, le délai de commutation n'ajoute qu'environ 2% de la latence totale. Il est donc **peu significatif** devant la propagation.

5.5 Store-and-forward contre cut-through

Deux modes de fonctionnement des nœuds intermédiaires sont fréquents :

— **store-and-forward** : le routeur attend le paquet complet avant de le retransmettre ;
— **cut-through** : le routeur commence à retransmettre dès qu'il a reçu suffisamment de bits.

Avec un message de taille M, un débit D, une distance totale L, une vitesse V et un seuil de N bits pour le cut-through :

Situation	Temps entre le premier bit émis et le dernier bit reçu
1 nœud store-and-forward	$2\frac{M}{D} + \frac{L}{V}$
3 nœuds store-and-forward	$4\frac{M}{D} + \frac{L}{V}$
1 nœud cut-through	$\frac{M}{D} + \frac{L}{V} + \frac{N}{D}$
3 nœuds cut-through	$\frac{M}{D} + \frac{L}{V} + 3\frac{N}{D}$

Exemple : avec $D = 1$ Gb/s, $L = 20$ km, $M = 5000$ bits, $N = 128$ bits

$$\frac{L}{V} = 100\ \mu s, \qquad \frac{M}{D} = 5\ \mu s, \qquad \frac{N}{D} = 0{,}128\ \mu s$$

On obtient :

— 1 nœud store-and-forward : 110 μs ;

— 3 nœuds store-and-forward : 120 μs ;

— 1 nœud cut-through : 105,128 μs ;

— 3 nœuds cut-through : 105,384 μs.

Le message essentiel est simple : le **cut-through limite l'effet cumulatif des sauts intermédiaires**, alors que le store-and-forward réapplique presque intégralement le temps d'émission à chaque lien.

5.6 Segmentation et pipeline

Supposons un message de 7,5 Mbits à transmettre de A vers B via deux routeurs, donc sur **3 liens**, chacun à 1,5 Mbit/s.

Sans segmentation. Le temps d'émission du message sur un lien vaut :

$$\frac{7{,}5 \times 10^6}{1{,}5 \times 10^6} = 5 \text{ s}$$

Donc :

— jusqu'au premier routeur : **5 s** ;

— jusqu'au destinataire final en store-and-forward : **15 s**.

Avec 5000 paquets de 1500 bits. Le temps d'émission d'un paquet sur un lien vaut :

$$\tau = \frac{1500}{1{,}5 \times 10^6} = 1 \text{ ms}$$

Le premier paquet met 3τ pour atteindre B, puis les suivants arrivent en **pipeline**, à raison d'un paquet par milliseconde. Pour q paquets et h liens, on obtient :

$$T = (h + q - 1)\tau$$

Ici, avec $h = 3$, $q = 5000$ et $\tau = 1$ ms :

$$T = (3 + 5000 - 1) \times 1 \text{ ms} = 5002 \text{ ms} = 5{,}002 \text{ s}$$

À retenir

La segmentation ne change pas la quantité totale de bits à envoyer, mais elle permet de **superposer les transmissions sur les différents liens**. C'est pour cela qu'elle réduit massivement la durée totale.

6 Focus application : transfert de fichiers et architecture client-serveur

Le PDF `Complement_FTP.pdf` rappelle une idée importante : un protocole applicatif ne flotte pas dans le vide. Il s'appuie sur :

— une **architecture client-serveur** ;
— des **sockets** pour utiliser TCP ou UDP ;
— des **messages applicatifs** qu'il faut définir clairement.

6.1 Ce que fait réellement un protocole de transfert

Un client de transfert de fichiers doit au minimum :

1. ouvrir une communication avec le serveur ;
2. indiquer **quel fichier** il souhaite ;
3. recevoir les données ;
4. détecter la **fin du transfert** et les erreurs éventuelles.

Cela montre bien la séparation des rôles :

— **l'application** définit les commandes et la logique métier ;
— **TCP** fournit un flot fiable d'octets si on choisit ce protocole ;
— **IP** et la couche liaison acheminent les paquets.

6.2 Serveur itératif ou serveur concurrent ?

Deux grands modèles apparaissent souvent dans les TP réseau :

Modèle	Principe	Conséquence
Serveur itératif	le serveur traite un client puis passe au suivant	simple, mais un autre client doit attendre
Serveur concurrent	plusieurs clients sont gérés en parallèle, par processus ou par pool	plus réaliste, meilleur débit de service

Cette distinction ne relève pas directement de la couche réseau, mais elle influence **l'expérience utilisateur**, la **charge du serveur** et la **qualité de service perçue**.

Point d'attention

Un bon raisonnement réseau ne s'arrête pas à "le paquet arrive". Il faut aussi penser à **qui parle à qui**, **combien de clients en parallèle** et **à quel coût en temps**.

7 Méthode générale pour résoudre les exercices du TD

Méthode

Pour un exercice de couches ou d'encapsulation

1. identifier la couche concernée ;
2. préciser l'unité manipulée : bit, trame, paquet, segment, message ;
3. distinguer le rôle local (liaison) du rôle global (réseau) ;
4. vérifier si l'on parle de **machine** ou de **processus**.

Pour un exercice de ports et de communication

1. vérifier le protocole de transport : TCP ou UDP ;
2. identifier les extrémités avec IP et ports ;
3. pour TCP, raisonner en termes de quadruplet ;
4. se demander si l'on parle d'une machine, d'un service ou d'une connexion.

Pour un exercice de performances

1. compter les liens et les nœuds intermédiaires ;
2. calculer $t_{\text{émission}}$ et $t_{\text{propagation}}$ séparément ;
3. décider si l'on est en store-and-forward, en cut-through ou en pipeline ;
4. faire attention à ce qui est demandé : premier bit reçu, dernier bit reçu, temps jusqu'au premier routeur ou temps de bout en bout.

8 Fiche de synthèse

À retenir

— Un réseau sert à rendre des **services** accessibles à distance.

— L'architecture en couches permet de maîtriser la complexité.

— Une communication complète mobilise l'application, le transport, le réseau, la liaison et le physique.

— Les premiers octets observés sur un lien sont en général ceux des **en-têtes**, pas ceux du fichier.

— Le champ **Protocol** d'IP indique si la charge utile contient TCP, UDP, ICMP, etc.

— Le routage choisit un **prochain saut** ; la livraison au bon programme repose sur les **ports**.

— La couche réseau peut exister sans couche transport : le routage reste possible.

— Le débit et la latence n'impactent pas toutes les applications de la même manière.

— En store-and-forward, chaque saut répète un temps d'émission complet ; en cut-through, le surcoût par saut est beaucoup plus faible.

— La segmentation permet le **pipeline** et améliore fortement les transferts multi-sauts.

Si vous savez expliquer ces dix points sans regarder vos notes, vous maîtrisez l'essentiel du TD.